Specialization B: Mutation Testing

Mikael Welander

Tools and configuration

Mutmut was used in this implementation. The ordinary integration test suite sends HTTP requests from a test container to the endpoints in the server container (or through localhost if running the tests locally). These tests cannot therefore run over the mutations that is under test. A dedication mutation container was therefore built with a modified test suite which calls the Flask app in process through app.test_client(). The modified test suite checks the same cases as the original.

The Setup is stored in `mutation/`:

- `Setup.cfg -` mutmut configuration. It mutates `validation.py` and `server.py`, copies dependencies into the mutation environment using `also_copy` and then runs the tests in `test/`
- `Dockerfile.mutation` and the mutation service in `docker-compose-yml` so mutation test runs can be started using `docker compose up`
- `init.sh` - runs mutmut and writes the result list and every survivor to `mutation_test_tesults/`

Results

The combined run over validation.py and server.py produced 329 mutants: 191 killed, 136 survived and 2 timed out. Of the 46 survivors in validation.py, 22 changed program logic and 24 only modified error-message strings.
Some interesting surviving mutants were however found for the validation.py test suite and source code:

ID	Mutation	Bug discovered
validate_signup__mutmut_11	- errors[field] = message + errors[field] = None	Survivor shows that the assertion test only checks that the error message exists (json keys), not their content.
validate_login__mutmut_6	+ if len(value) > USERNAME_MAX_LENGTH: + if len(value) >= USERNAME_MAX_LENGTH:	Mutant survives even though the limit was changed. After checking there are no boundary assertion tests for this which is why it survived.
sanitize_filename__mutmut_21	- if len(name) > UPLOAD_FILENAME_MAX_LENGTH: + if len(name) >= UPLOAD_FILENAME_MAX_LENGTH:	Similar to the validate login mutant, i discovered that there are no boundary assertions for this
validate_password__mutmut_8	- ord(c) == 127 + ord(c) == 128	By breaking the condition and the mutant still surviving I realized that this condition is not covered in the test suite assertions.

validate_password__mutmut_14	Control char check: - ord(c) == 127 + ord(c) == 128	Discovered that this condition only covers and checks ASCII controls. Not Unicode C1 (128-159).

Fixes

validate_password__mutmut_14 & validate_password__mutmut_8:
The fix implemented due to these mutants was to modify the source code condition to check for both ASCII and Unicode C1 control characters.
I also added additional assertion tests for ascii and unicode control characters, which was not previously tested in the original test suite.
389c4c6751b507855c77f410b99f31391c8637bb

validate_signup__mutmut_11:
The fix that was implemented changed so that the assertion check for the expected string in the error key instead of just the error dict itself, which can be replicated for more assertion tests with different arguments. Better traceback if code changes.
2aead1f77d8d8092020f8a5ba5331454f9a9600b

validate_login__mutmut_6:
The fix that was implemented added a boundary edge case test both for minimum and maximum username length. Any changes to the condition in the source code is now catched by the tests and the mutant in question should not survive.
E94cab189be7b3fc4a45bb2a5522be37e693d1a8

sanitize_filename__mutmut_21:
The fix that was implemented was boundary unit test to test a maximum filesize upload size.
5b3f74ca0f488fc01e8b15a7731dd6b414723fea

Second run

After these fixes, a second mutation test run shows a total of 32 survivors, where 11 of them are logical mutations and 21 are message string mutations.